

List.h

```
link NEWnode(Item val, link next);
void LISTinshead(LIST l, Item val);
link LISTinstail(link head, Item val);
link LISTinstail(link head, Item val);
ITEM LISTsearch(link head, Key k);
link LISTdelhead(link h);
ITEM LISTextrheadP(link *hp);
link LISTdelkey(link h, Key k);
link LISTdelkeyR(link x, Key k);
ITEM LISTextrkeyP(link *x, Key k);
link LISTsortins(link h, ITEM item);
ITEM LISTsortsearch(link h, Key k);
link LISTsortdel(link h, Key k);
void LISTshow(link h);
void LISTfree(link h);
```

BST.h

```
BST BSTinit();
void BSTfree (BST bst);
int BSTcount (BST bst);
int BSTempty (BST bst);
Item BSTsearch(BST bst, Key k);
Item BSTmin(BST bst);
Item BSTmax(BST bst);
void BSTinsert_leafR(BST bst, Item x);
void BSTinsert_leafI(BST bst, Item x);
void BSTinsert_root(BST bst, Item x);
void BSTvisit (BST bst, int strategy);
link rotR(link h);
link rotL(link h);
link partR (link h, int r);
void BSTdelete (BST bst, Key k);
Item BSTselect (BST bst, int r);
Item BSTsucc (BST bst, Key k);
Item BSTpred (BST bst, Key k);
```

IBST.h

```
void IBSTinit(IBST ibst);
void IBSTfree (IBST ibst);
void BSTinsert (IBST ibst, Item x);
void IBSTdelete (IBST ibst, Item x);
Item IBSTsearch (IBST ibst, Item x);
Int IBSTcount (IBST ibst);
int IBSTempty (IBST ibst);
void IBSTvisit (IBST ibst, int strategy);
```

PQ.h

```
PQ PQinit(int maxN);
void PQfree (PQ pq);
int PQempty(PQ pq);
void PQinsert(PQ pq, Item val);
Item PQextractMax(PQ pq);
Item PQshowMax(PQ pq);
void PQdisplay(PQ pq);
int PQsize(PQ pq);
void PQchange(PQ pq, Item val);
void PQchange(PQ pq, int pos, Item val);
```

Heap.h

```
Heap HEAPinit(int maxN);
void HEAPfill(Heap h, Item val);
void HEAPSsort(Heap h);
void HEAPdisplay(Heap h);
void HEAPfree(Heap h);
void HEAPIfy(Heap h, int i);
void HEAPbuild(Heap h);
int PARENT(int i);
int RIGHT(int i);
int LEFT(int i);
```

ST.h

```
ST STinit(int maxN);
void STdisplay(ST st);
int STsize(int N);
int STinsert(ST st, Item val);
int STcount(ST st);
int STempty(ST st);
int STselect(ST st, int r);
int STcount (ST st);
void STinsert (ST st, Item val);
int STgetindex(ST tabella, ITEM item);
Item STsearch(ST st, Key k);
int STsearch (ST st, Key k);
Key STsearchByIndex (ST st, int id);
void STdelete(ST st, Key k);
void STfree(ST st);
void STdisplay (ST st);
int hashU(char *v, int M);
int hash (Key k, int M);
int full((ST st, int i);
void STchangePrio (ST st, Item val, int i);
```

Queue.h

```
QUEUE QUEUEinit(int maxN);
int QUEUEempty(QUEUE q);
```

```
void QUEUEput(QUEUE queue, Item val);
Item QUEUEget(QUEUE q);
```

Graph.h

```
Graph GRAPHinit(int V);
void GRAPHfree(Graph G);
Graph GRAPHload(FILE *fin);
void GRAPHstore(Graph G, FILE *fout);
void GRAPHgetIndex(Graph G, char*label);
void GRAPHinsertE(Graph G, int id1, int id2, int wt);
void GRAPHremoveE(Graph G, int id1, int id2);
void GRAPHshow(Graph G);
void GRAPHedges(Graph G, Edge *a);
void insertE(Graph G, Edge e);
void removeE(Graph G, Edge e);
int randV(Graph G);
Graph GRAPHrand1(Graph G, int V, int E);
Graph GRAPHrand2(Graph G, int V, int E);
int GRAPHpath(Graph G, int id1, int id2);
void GRAPHpathH (Graph G, int id1, int id2);
void GRAPHbfs(Graph G, int id);
void GRAPHdfs(Graph G, int id);
int GRAPHscc(Graph G);
int GRAPHcc(Graph G);
Graph reverse(Graph G);
void GRAPHmstK(Graph G);
void GRAPHmstP(Graph G);
void GRAPHspD(Graph G, int id);
void GRAPHspBF(Graph G, int id);
```

Arco.h

```
Edge EDGEcreate(int v, int w, int wt);
Edge EDGEalloc(int v, int w, int wt);
```

IterativeSort.h

```
void BubbleSort(Item A[], int N);
void OptBubbleSort(Item A[], int N);
void SelectionSort(Item A[], int N);
void InsertionSort(Item A[], int N);
void ShellSort(Item A[], int N);
void CountingSort(Item A[], Item B[], int C[], int N, int k);
```

RecursiveSort.h

```
void QuickSort (Item *A, int N);
void QuickSortR(Item *A, int l, int r);
int partition(Item *A, int l, int r);
void MergeSort (Item *A, int N);
void MergeSortR(Item *A, Item * B, int l, int r);
void Merge(Item *A, Item *B, int l, int q, int r);
void BottomUpMergeSort (Item *A, int N);
```